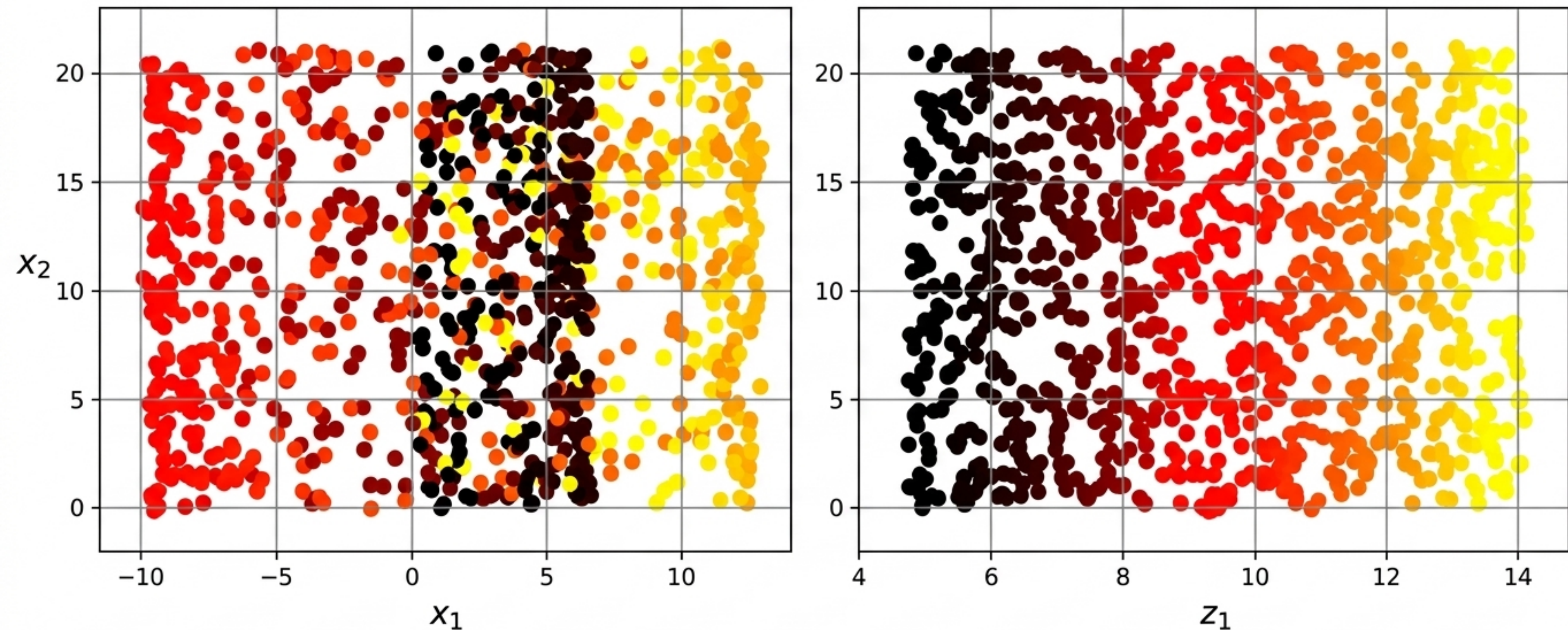
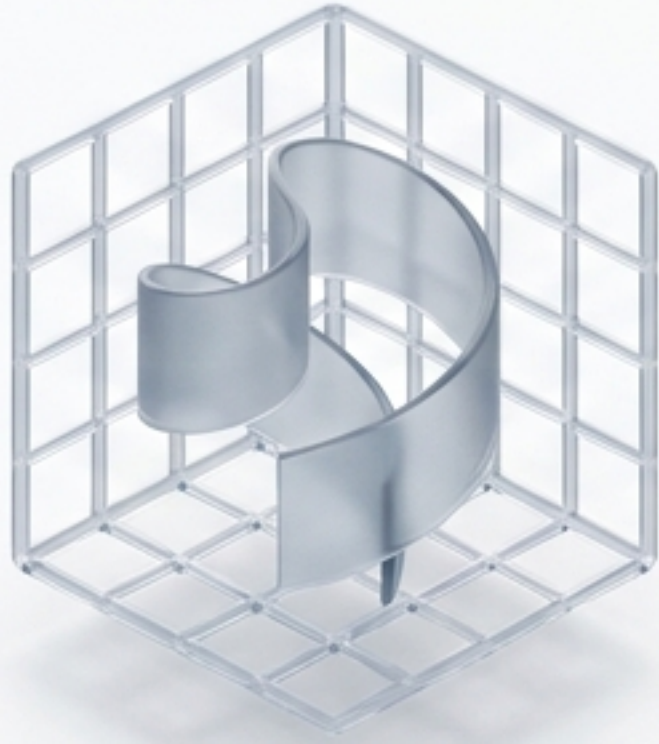


Làm dẹt (Phép chiếu) vs Mở cuộn (Đa tạp)



Hình 8-5. Chiếu vuông góc làm dẹt cuộn (trái) - sai lầm; Mở cuộn (phải) - chính xác

Giả định đa tạp (Manifold Hypothesis)



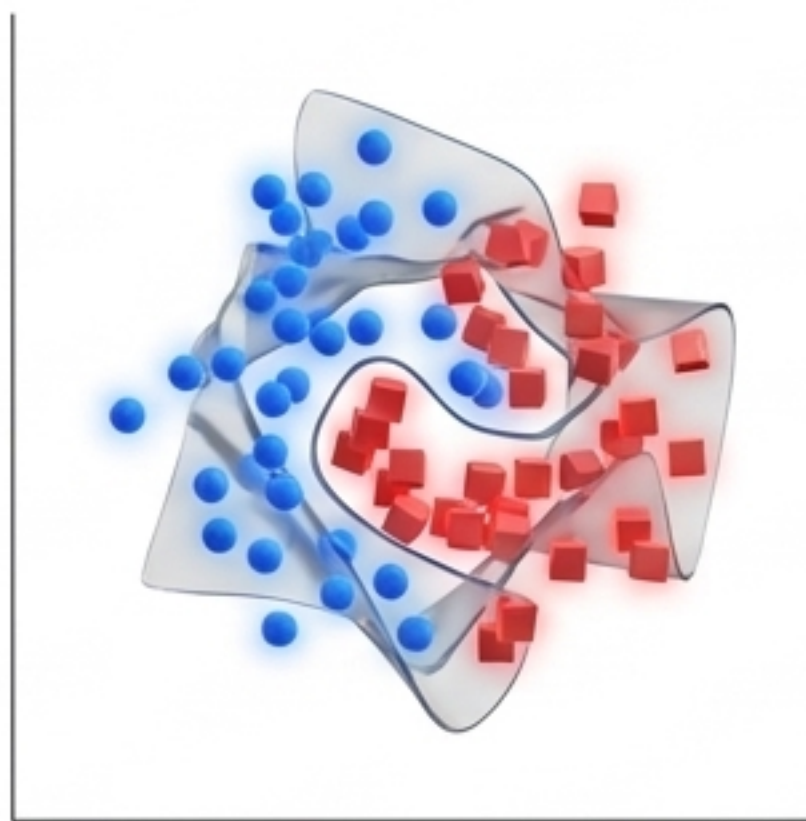
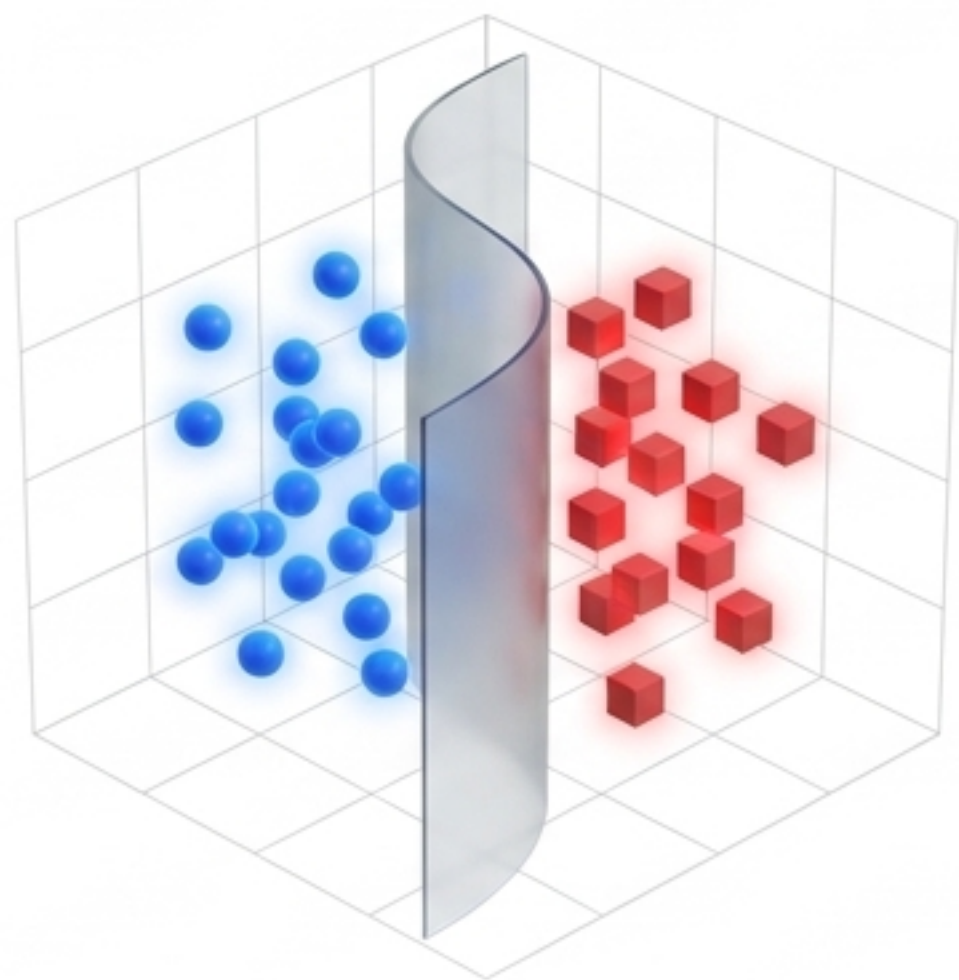
Đa tạp 2D: Là một hình 2D có thể bị uốn cong, xoắn lại trong một không gian có chiều cao hơn. Tổng quát, một đa tạp d -chiều nằm trong không gian n -chiều ($d < n$).



Giả định đa tạp: Hầu hết các tập dữ liệu thực tế có chiều cao đều nằm gần một đa tạp có chiều thấp hơn nhiều.

Các tác vụ học máy (như phân loại/hồi quy) thường sẽ đơn giản hơn nếu được biểu diễn trong không gian đa tạp (sau khi được mở cuộn).

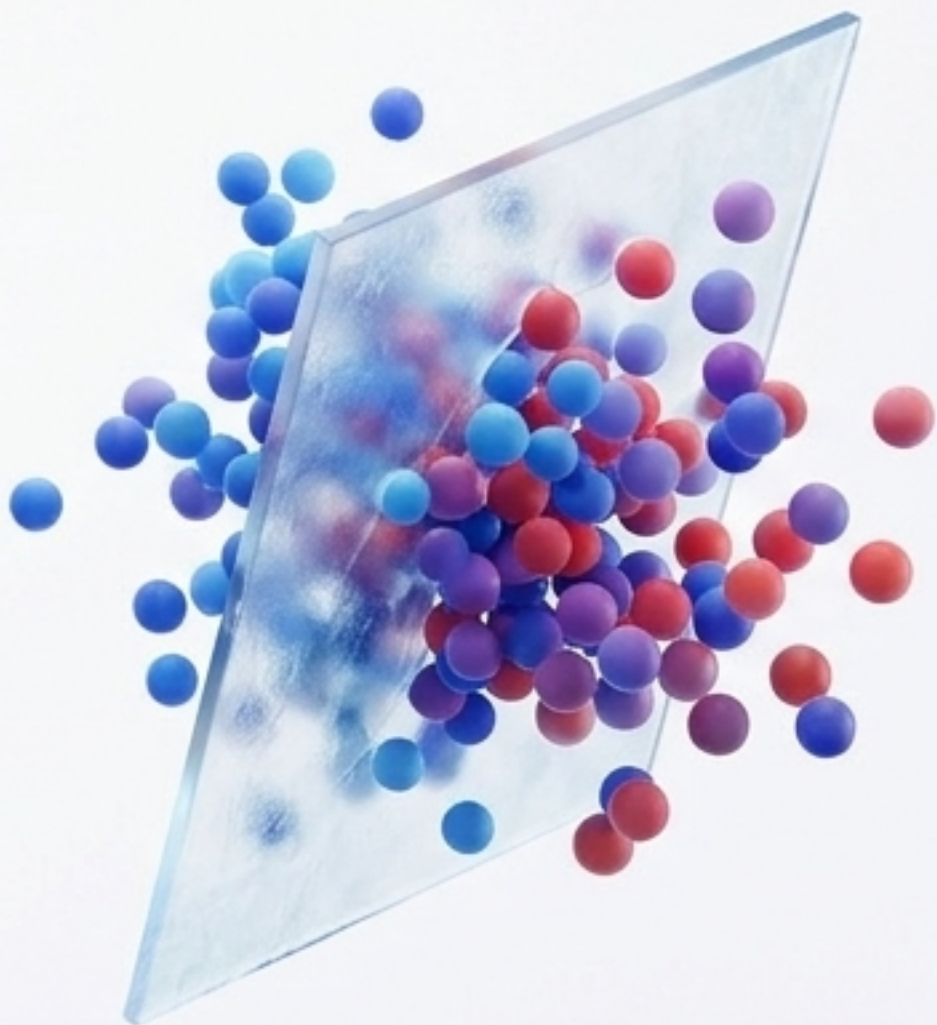
Lưu ý về Giả định đa tạp



Hình 8-6. Đường biên quyết định không phải lúc nào cũng đơn giản hơn với không gian chiều thấp.

Phân tích thành phần chính (PCA)

Phân tích thành phần chính (Principal Component Analysis - PCA) cho đến nay là thuật toán giảm chiều phổ biến nhất. Nó là một phương pháp thuộc nhóm Phép chiếu (Projection).



1

Đầu tiên xác định siêu phẳng (hyperplane) nằm gần dữ liệu nhất.



2

Sau đó chiếu toàn bộ dữ liệu lên mặt phẳng đó.



Bảo toàn phương sai (Preserving Variance)

MỤC TIÊU

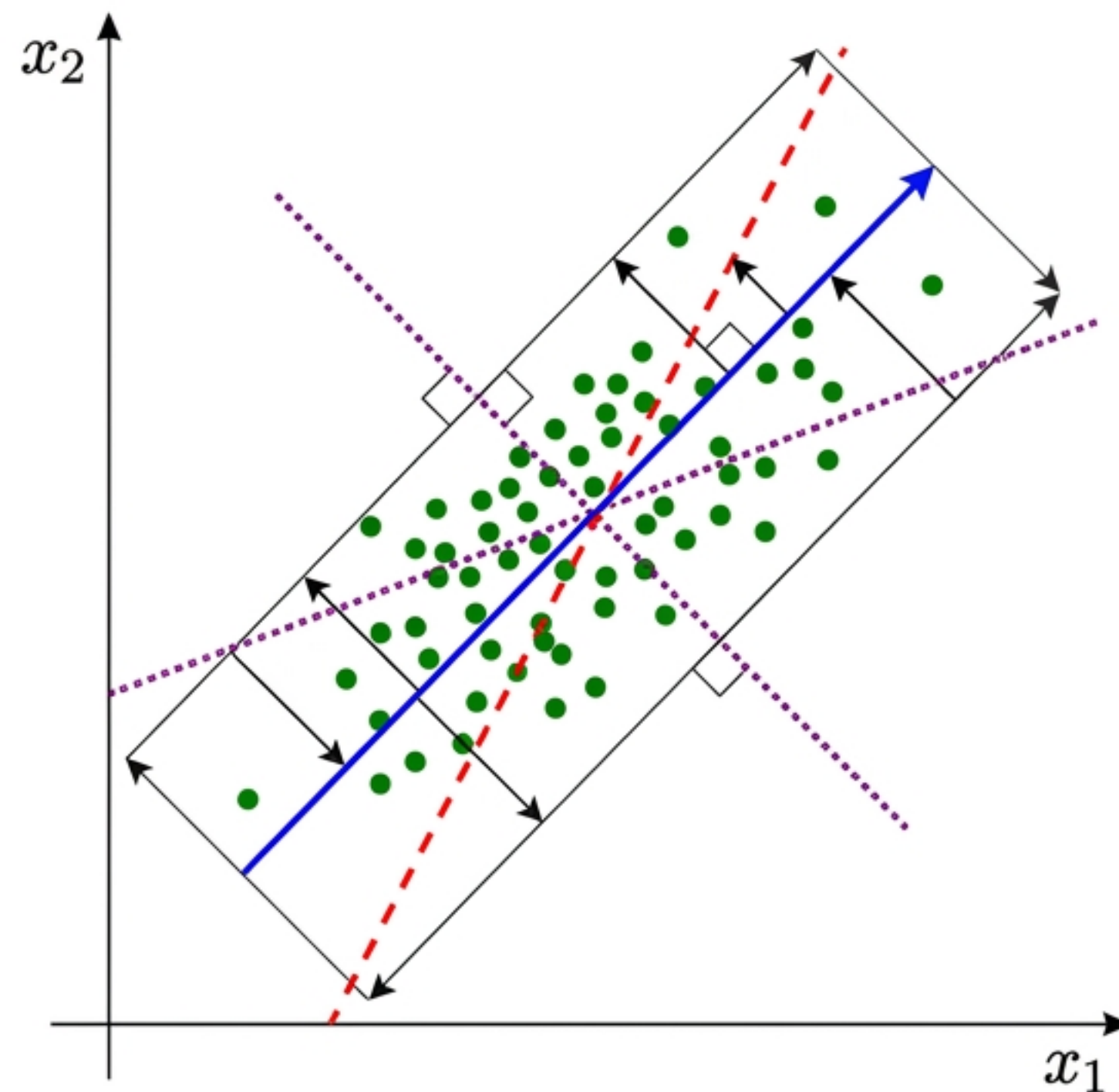
- Trước khi chiếu dữ liệu, cần phải chọn một siêu phẳng phù hợp.

NGUYÊN TẮC

- PCA sẽ chọn siêu phẳng (hoặc trục) sao cho bảo toàn được lượng phương sai lớn nhất của dữ liệu. Giúp giữ lại nhiều thông tin nhất có thể.

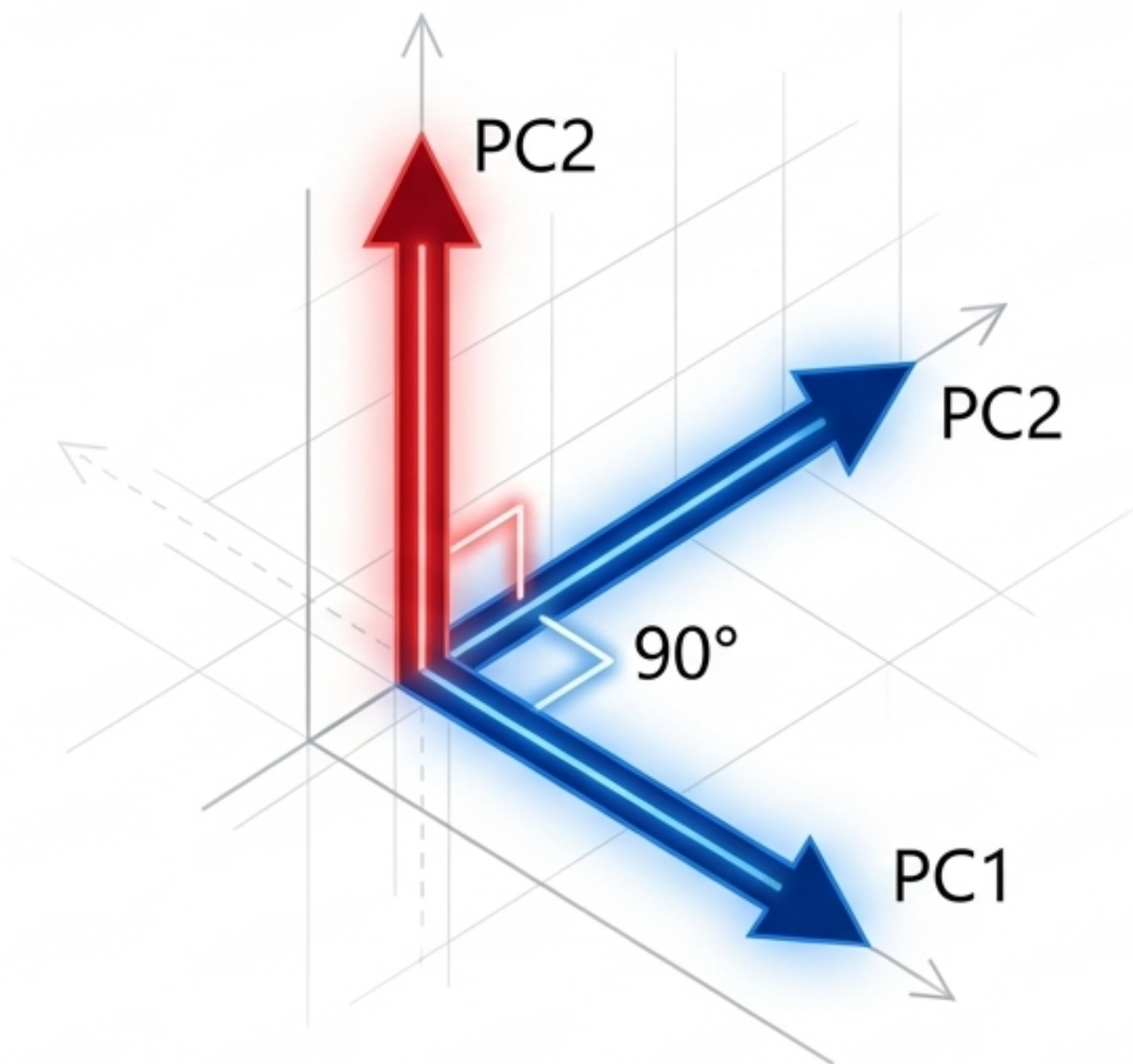
TÍNH TƯƠNG ĐƯƠNG

- Việc này cũng tương đương với việc chọn trục làm giảm thiểu khoảng cách bình phương trung bình giữa tập dữ liệu gốc và điểm chiếu.



Các thành phần chính (Principal Components - PC)

Lựa chọn không gian con để chiếu (Hình 8-7. Phép chiếu lên đường liền nét bảo toàn toàn phương sai tối đa).



- 1 **PC1:** Tìm trục có phương sai lớn nhất trong tập huấn luyện.
- 2 **PC2:** Tìm trục thứ 2 trực giao (vuông góc) với trục 1 và chứa nhiều phương sai nhất từ phần còn lại.
- 3 **PC_n:** Quá trình tiếp diễn tối đa bằng số chiều của không gian gốc.

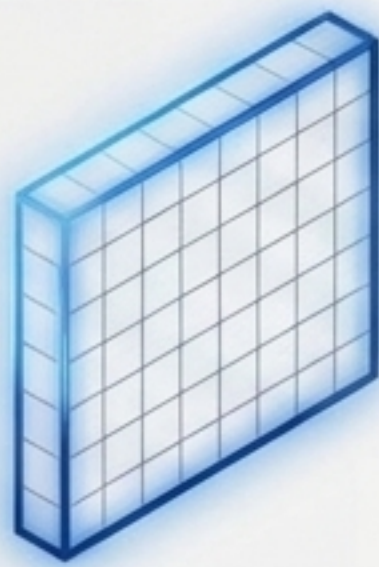
Trục thứ i được gọi là **thành phần chính thứ i** của dữ liệu.

Toán học PCA: Phân tách giá trị số ít (SVD)

Để tìm tất cả các thành phần chính, ta sử dụng kỹ thuật phân tách ma trận tiêu chuẩn gọi là **Phân tách giá trị số ít** (Singular Value Decomposition - SVD).

SVD phân tách ma trận huấn luyện $\mathbf{X}$ thành ba ma trận:

$$\mathbf{X} = \mathbf{U} \cdot \mathbf{\Sigma} \cdot \mathbf{V}^T$$



Ma trận $\mathbf{V}^T$ chứa tất cả các thành phần chính cần tìm (các vector đơn vị trực giao).

Công thức $\mathbf{V}^T$:
$$\mathbf{V}^T = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & \cdots & | \end{pmatrix}^T$$

Tính toán PCA bằng NumPy (Phần 1)

LOGIC



Dùng hàm `svd()` của NumPy. Lưu ý **quan trọng**: PCA giả định rằng dữ liệu được căn giữa (**centered**) xung quanh gốc tọa độ. Cần trừ đi trung bình trước khi chạy SVD.

SYNTAX

```
import numpy as np

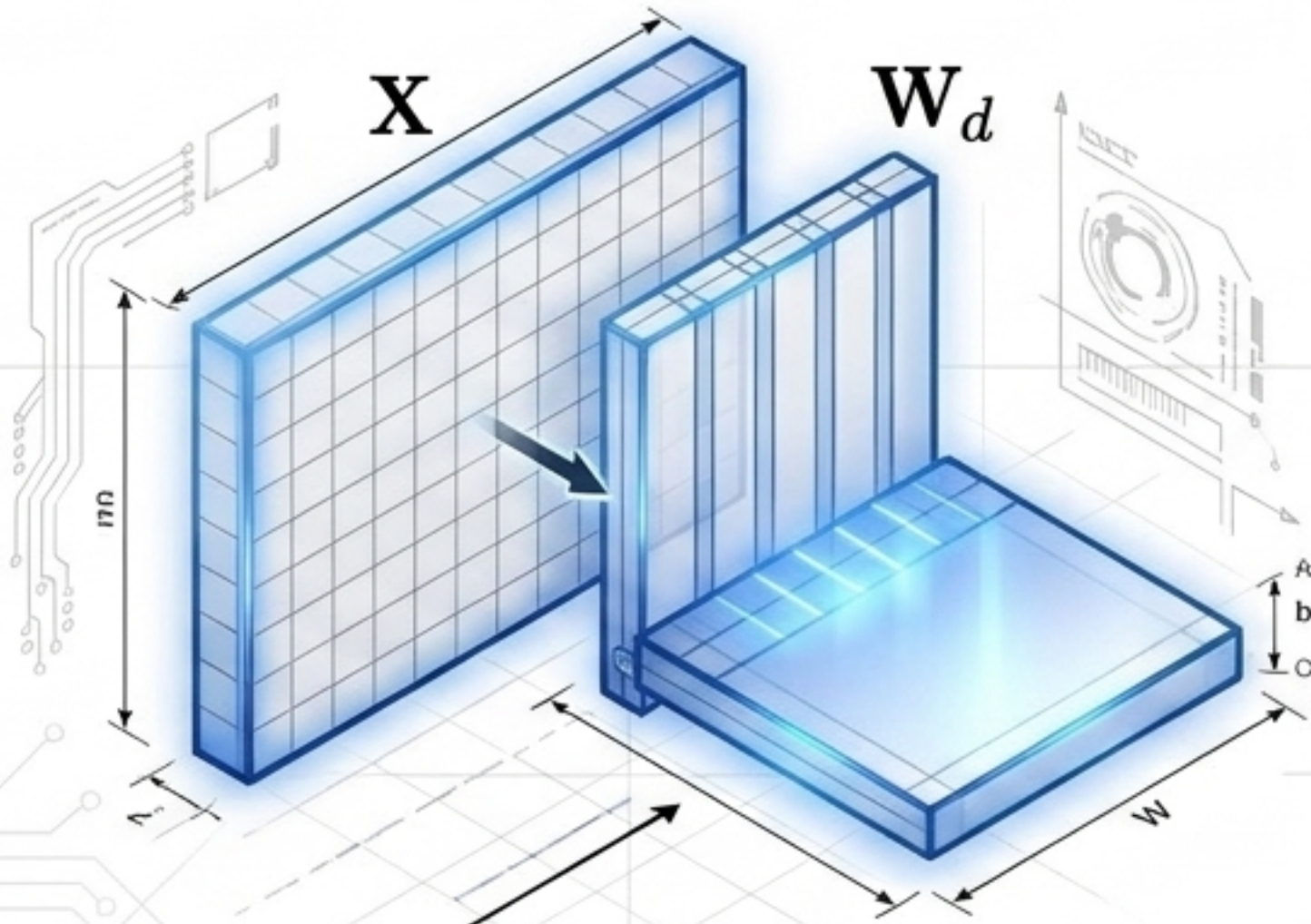
# Giả sử X là tập dữ liệu của bạn
X_centered = X - X.mean(axis=0)

# Phân tách giá trị số ít
U, s, Vt = np.linalg.svd(X_centered)

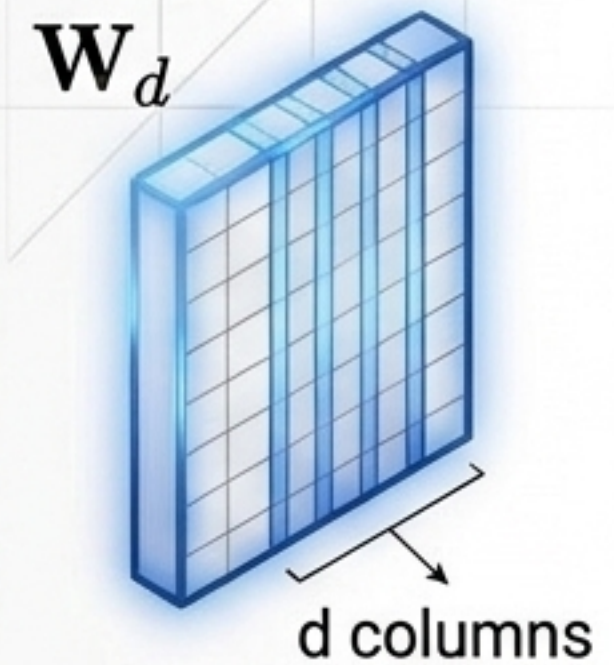
c1 = Vt[0] # Thành phần chính thu 1 (PC1)
c2 = Vt[1] # Thành phần chính thu 2 (PC2)
```


Chiếu dữ liệu xuống d chiều

Sau khi có các thành phần chính (PC), ta có thể giảm kích thước không gian bằng cách chiếu tập dữ liệu lên siêu phẳng tạo bởi d thành phần chính đầu tiên ($d < n$).



Ma trận W_d được định nghĩa là ma trận chứa d cột đầu tiên của V (hay d hàng đầu tiên của V^T chuyển vị).



Phép chiếu: $X_{d-proj} = X \cdot W_d$

Thực hiện phép chiếu bằng NumPy

LOGIC

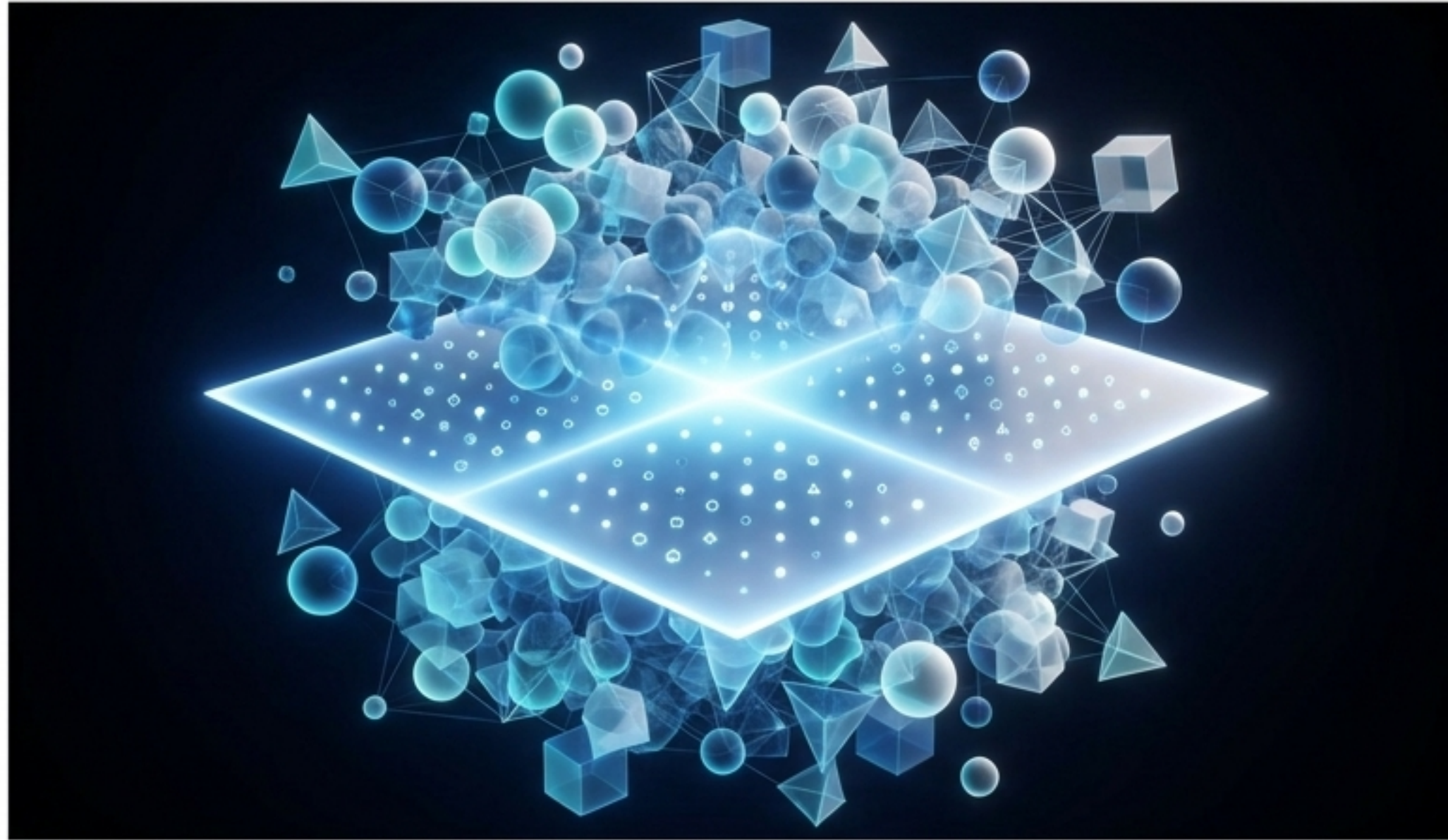


Đoạn mã sau chiếu tập huấn luyện lên **mặt phẳng 2D** định nghĩa bởi hai PC đầu tiên.

SYNTAX

```
# Chọn 2 thành phần chính đầu tiên và chuyển vị  
# Chọn 2 thành phần chính đầu tiên và chuyển vị  
W2 = Vt[:2].T  
  
# Thực hiện phép nhân ma trận (phép chiếu)  
X2D = X_centered @ W2
```


Kết quả: Dữ liệu 2D sau giảm chiều



Bây giờ X2D chính là dữ liệu 2D đã được giảm chiều, bảo toàn được tối đa lượng phương sai có thể từ tập X ban đầu.

Sử dụng Scikit-Learn cho PCA

Trong thực tế, Scikit-Learn cung cấp lớp PCA tự động xử lý SVD và quan trọng nhất là tự động căn giữa dữ liệu. (Không cần trừ đi trung bình thủ công).



```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)  
X2D = pca.fit_transform(X)
```

```
# Lay cac vector thanh phan chinh  
print(pca.components_)
```

`pca.components_` chứa các thành phần chính đã tìm thấy.

Tỷ lệ phương sai giải thích (Explained Variance Ratio)



Đây là một chỉ số quan trọng cho biết tỷ lệ phương sai của tập dữ liệu phân bố dọc theo từng thành phần chính. Xem kết quả này qua biến `explained_variance_ratio_`.

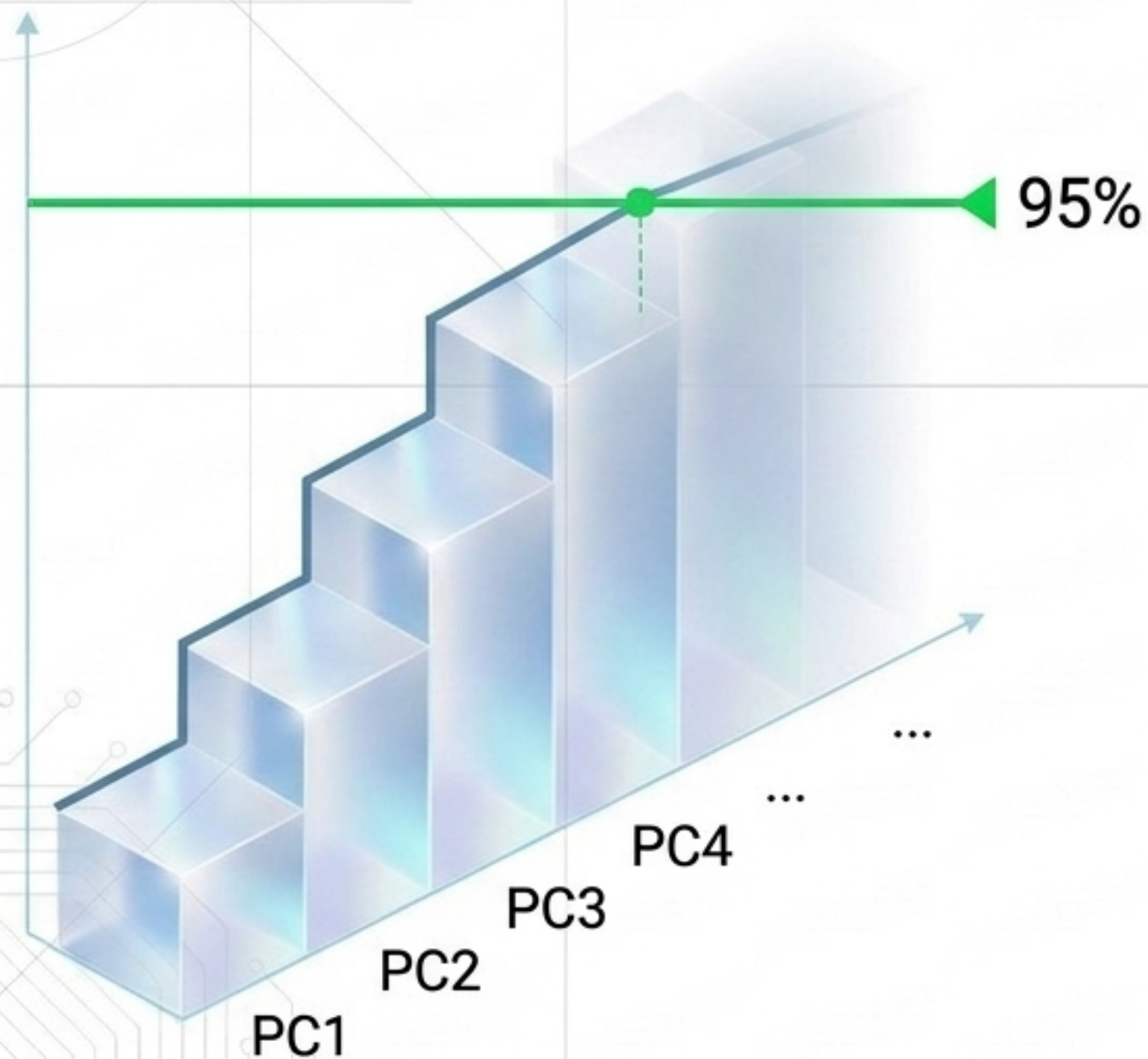
Đánh giá tỷ lệ phương sai giải thích

```
>>> pca.explained_variance_ratio_  
array([0.7578477 , 0.15186921])
```

- Ý nghĩa: $\approx 76\%$ phương sai của dữ liệu nằm dọc theo trục PC1.
 - ◆ $\approx 15\%$ nằm trên trục PC2.
- Tổng cộng: PC1 và PC2 giải thích được $> 90\%$ phương sai.

Kết luận: Điều này ngụ ý rằng các PC tiếp theo (PC3, PC4...) chứa **rất ít thông tin hữu ích** (chỉ còn khoảng 9%).

Chọn số chiều phù hợp như thế nào?



Quy tắc chung:

Trừ khi giảm xuống 2D hoặc 3D để vẽ biểu đồ trực quan, hãy chọn số chiều d sao cho tổng phương sai được giữ lại đủ lớn (ví dụ: 95%).

Nghĩa là, bạn sẽ thêm lần lượt PC1, PC2, PC3... cho đến khi tổng tỷ lệ phương sai giải thích vượt qua mức 95%.